IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA
(Deemed To Be University)

YENEPOYA UNIVERSITY

HIGH LEVEL DESIGN (HLD) DOCUMENT
ON
ANOMALY DETECTION FOR HEALTHCARE TRANSACTIONS

Student Name: K Mohammad Irfan Hussain

Registration No: 23BSAMD012

Campus ID: 26998

Project Code: PRJN26-229

Industry Mentor: Sumit Kumar Shukla

Guided By: Ms. Fathimath Raheema

Table of Contents

1. Introduction……3

1.1 Scope of the document……3

1.2 Intended Audience……3

1.3 System overview……3

2. System Design……4

2.1 Application Design……4

2.2 Process Flow……4

2.3 Information Flow……4

2.4 Components Design……5

2.5 Key Design Considerations……5

2.6 API Catalogue……5

3. Data Design……7

3.1 Data Model……7

3.2 Data Access Mechanism……8

3.3 Data Retention Policies……8

3.4 Data Migration……8

4. Interfaces……8

5. State and Session Management……10

6. Caching……11

7. Non-Functional Requirements……11

7.1 Security Aspects……11

7.2 Performance Aspects……11

8. References……11

1. Introduction

The Healthcare Fraud Anomaly Detection System is an AI-driven platform built to identify suspicious billing and operational patterns in healthcare claims. The system evaluates provider behaviors against historical baselines to classify transactions into distinct operational risk zones: Normal, Suspicious, and High Risk.

1.1 Scope of the document

This High-Level Design (HLD) outlines the macro-level architecture, user interfaces, database schema, and integration flows of the system. It defines the structural boundaries of the application and how various tiers communicate.

Note: Low-level programmatic specifics (e.g., machine learning logic, JSON payload contracts, temporal sequence math) are intentionally excluded from this document as they are explicitly detailed in the accompanying Low-Level Design (LLD).

1.2 Intended Audience

This document is intended for:

- **Academic Evaluators and Instructors:** To assess the architectural soundness and design methodology of the project (PRJN26-229).
- **Industry Mentors:** To review the system's macro-architecture and ensure its alignment with real-world healthcare fraud detection standards.
- **Software Developers:** To serve as the primary implementation blueprint for building the decoupled backend APIs and frontend dashboards.

1.3 System overview

The system operates primarily as a runtime screening mechanism for claims. Administrators interact with a web dashboard to submit single claims or batch audit files. The core engine accesses historical records, processes the incoming claims, and returns an actionable verdict (auto-clear, manual review, or audit escalation) paired with natural-language explainability.

2. System Design

2.1 Application Design

The application relies on a decoupled, 4-tier architecture:

- **Presentation Tier (Streamlit):** Manages user inputs and renders visual outputs (charts, risk zones).
- **Application Tier (FastAPI):** Acts as the central router, managing HTTP requests and validation.
- **ML Engine Tier (Scikit-Learn/Keras):** Contains the logic for anomaly scoring (detailed in LLD).
- **Data Tier (SQLite):** Persistent storage for all historical healthcare transactions.

2.2 Process Flow

The high-level operational flow follows these steps:

1. User submits a claim or CSV batch via the Presentation Tier.
2. The Application Tier validates the payload structure.
3. Historical context for the corresponding provider is fetched from the Data Tier.
4. The ML Engine evaluates the claim against the provider's historical baseline.
5. A final risk zone (Normal: ≤ 1.30, Suspicious: 1.30 - 1.85, High Risk: ≥ 1.85) is assigned.
6. Results, along with textual explainability, are routed back and rendered on the dashboard.

2.3 Information Flow

Information flows linearly via HTTP REST calls between the UI and Backend, and via internal queries between the Backend and Database. External files (CSV batches) are parsed into internal data frames at the UI level before being transmitted to the Application Tier to ensure network payloads remain strictly data-oriented.

2.4 Components Design

The architecture ensures component independence. The FastAPI backend is entirely agnostic to the Streamlit frontend. If the presentation tier is replaced in the future (e.g., with React or Angular), the backend API, ML Engine, and Data tier will function without modification.

2.5 Key Design Considerations

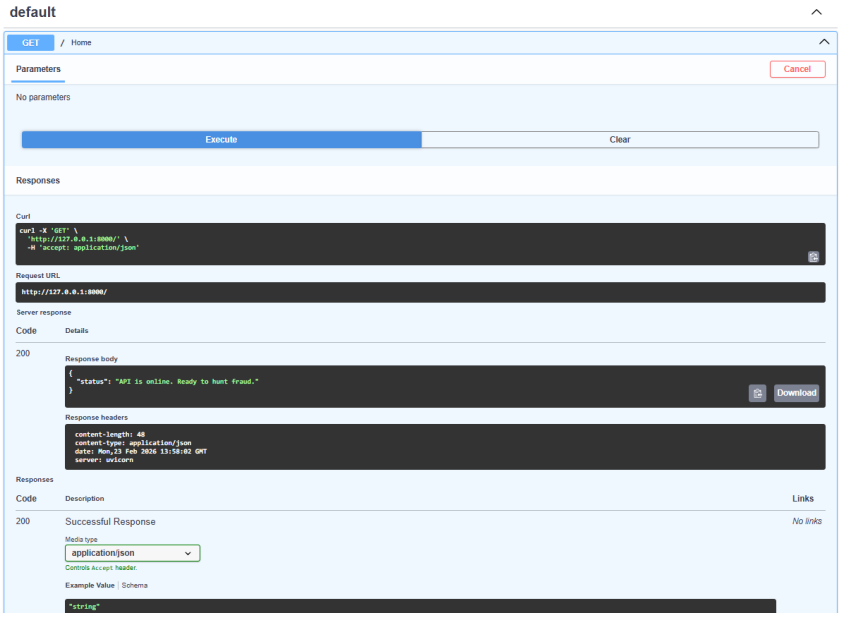
A major design consideration is **Fallback Execution**. If a provider lacks sufficient historical data to run advanced temporal analysis, the system must elegantly fall back to static anomaly detection to ensure no claim fails processing due to a "cold start" provider profile.

2.6 API Catalogue

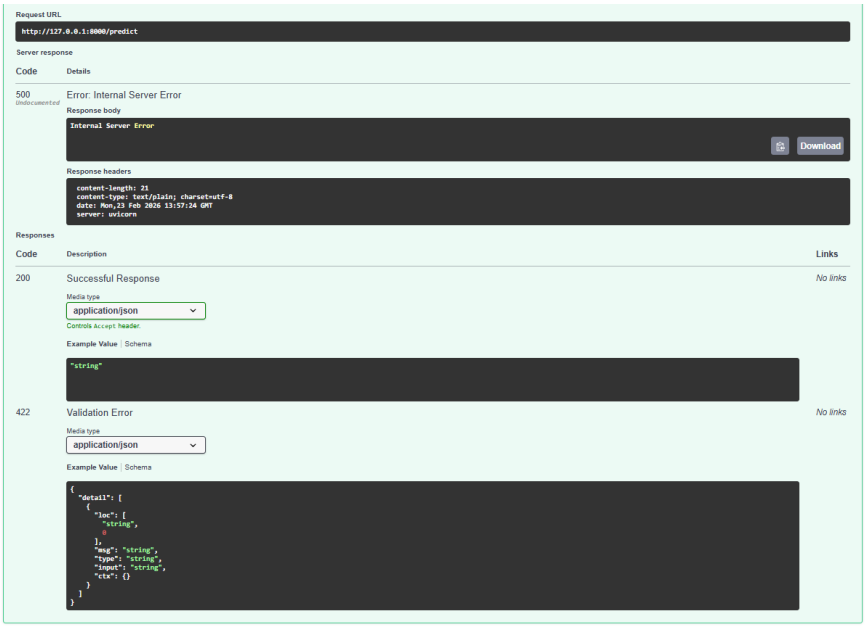
The backend exposes specific RESTful endpoints to facilitate integration:

Endpoint	Type	Description
`GET /`	Health Check	Verifies server uptime and readiness.
`POST /evaluate_transaction`	Single Inference	Accepts one claim, triggers scoring, and returns UI rendering data including SHAP visual metrics.
`POST /evaluate_batch`	Batch Inference	Accepts multiple claims for high-throughput processing, bypassing visual explainability to minimize latency.

API Health Check (Success)



API Error Handling Architecture



The API robustly handles server faults and data validation (HTTP 422/500).

3. Data Design

3.1 Data Model

The persistent data model relies on a singular core table, `Healthcare_transactions`, which stores the raw attributes of historical claims before algorithmic processing.

Field	Type	Description
transaction_id	Integer	Unique 14-digit transaction identifier
patient_id / provider_id	String	Unique alphanumeric identifiers for patient and facility
service_start / end	DateTime	Chronological boundaries of the healthcare service
billing_amount	Float	Total billed amount for the claim
deductible_context / coinsurance	Float	Patient financial responsibility parameters
diagnosis_context / primary_procedure	String	Standardized clinical coding for the visit
Chronic Condition Flags	Binary	0/1 flags indicating presence of severe ongoing conditions

3.2 Data Access Mechanism

During live inference, the system retrieves only the historical rows associated with the incoming provider_id instead of loading the full transaction table. This reduces unnecessary data loading and keeps runtime inference focused on the relevant provider history.

3.3 Data Retention Policies

Given the academic scope of this project, data retention is managed locally via the SQLite file mechanism. Historical transaction records and evaluation CSVs are retained locally for development and testing without automated archival schedules.

3.4 Data Migration

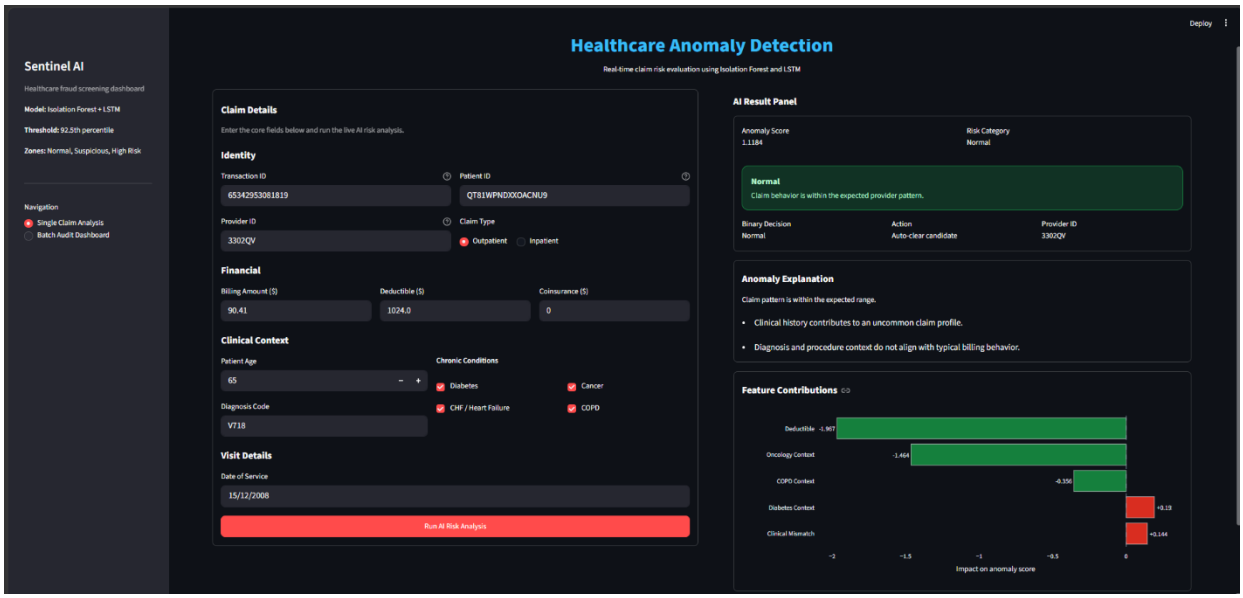
The current implementation does not use an automated migration framework. Database creation, schema preparation, and historical data loading are performed manually before training and inference execution.

4. Interfaces

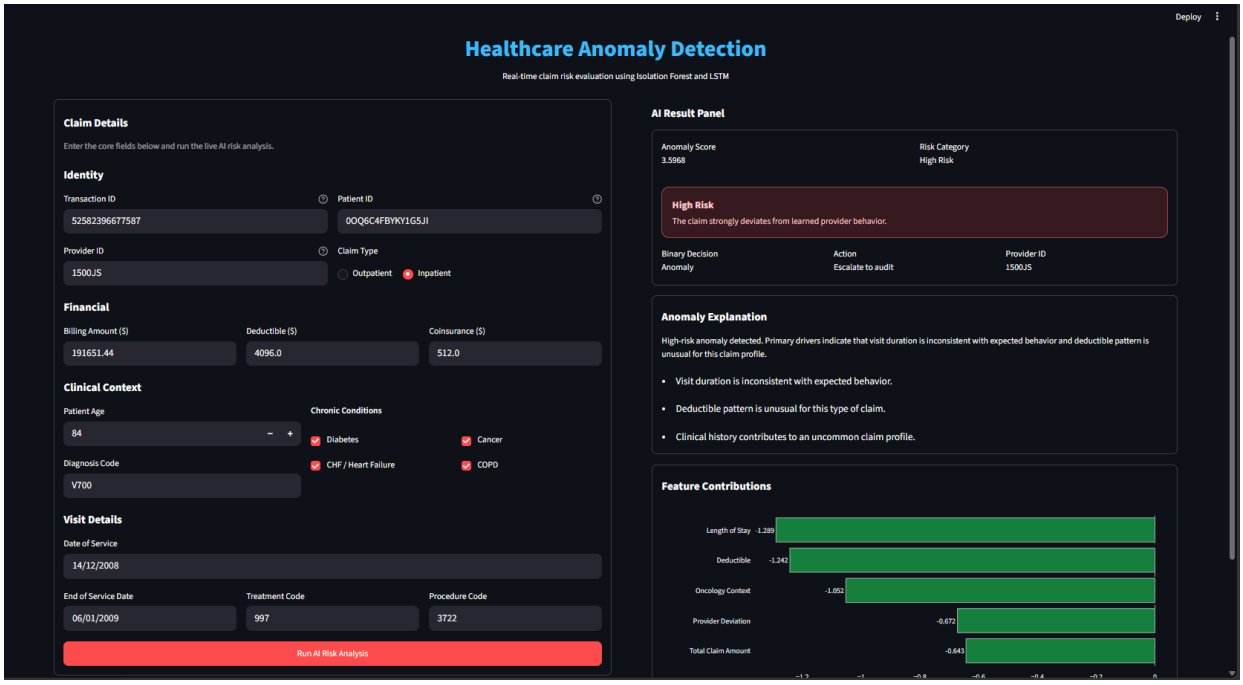
The system provides two primary user interfaces depending on the audit requirement.

Single Claim Analysis Dashboard: Designed for manual review, splitting inputs into Identity, Financial, Clinical Context, and Visit Details. Outputs provide deep explainability.

Single Claim Analysis: Normal Behaviour

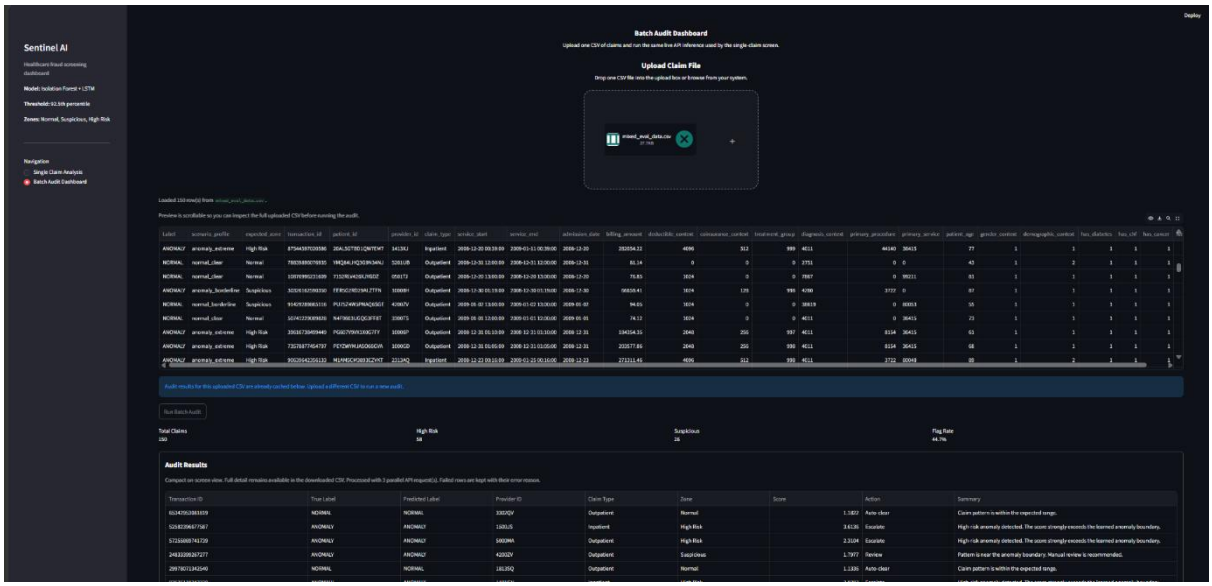


Single Claim Analysis: High-Risk Alert



Batch Audit Dashboard: Designed for high-volume end-of-day operations, returning aggregate metrics and downloadable tables.

Batch Audit Dashboard Interface



5. State and Session Management

The Application Tier (Backend) operates completely statelessly. Every incoming HTTP request is evaluated independently without relying on memory from previous requests. The Machine Learning models are loaded into RAM once at startup to act as static evaluators.

State management is pushed entirely to the Presentation Tier (Frontend). The Streamlit UI utilizes browser-level session state to preserve the results of expensive operations (like a 1000-row batch upload), preventing the system from wiping data if the user simply changes a dashboard tab.

6. Caching

To maximize inference throughput, the system caches static ML artifacts (model weights, thresholds, and scalers) in application memory on server boot. This eliminates file I/O bottlenecks during live REST requests. External distributed caching systems (like Redis) are not utilized as the current local caching strategy successfully meets response time objectives.

7. Non-Functional Requirements

7.1 Security Aspects

- **Input Validation:** All data boundaries are strictly enforced by backend Pydantic models, rejecting malformed JSON payloads before algorithmic execution.
- **Microservice Isolation:** Deployed via Docker, the backend ML container operates on an internal network, meaning endpoints cannot be directly accessed by public internet traffic bypassing the frontend gateway.

7.2 Performance Aspects

The system guarantees high throughput for multi-row operations by bypassing visual rendering overhead during batch endpoints. In batch scenarios, the engine is optimized to achieve rapid processing times while maintaining reliable classification. Failed batch rows are captured and logged, rather than crashing the overall process.

8. References

- **Low-Level Design (LLD):** *Anomaly Detection for Healthcare Transactions (PRJN26-229)* — Contains micro-architecture and ML feature engineering specifics.
- **Algorithm Literature:** Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). *Isolation Forest*. IEEE International Conference on Data Mining.
- **Algorithm Literature:** Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory (LSTM)*. Neural Computation.
- **Compliance & Security:** Internal guidelines for Healthcare PII (Personally Identifiable Information) masking and secure data handling.